PRACTICAL LEARNING GUIDE

LangSmith for an Existing RAG

From one live trace to governed datasets, repeatable experiments, evaluator feedback, comparison, and production learning loops

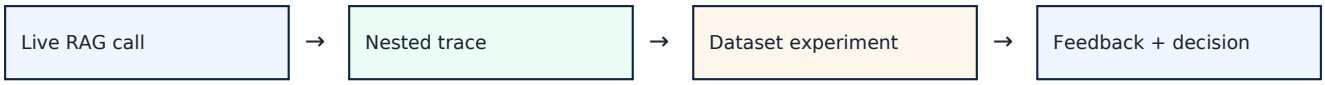
What this guide uses
The existing payroll-MFA RAG, two synthetic policy documents, and the eight governed MFA golden cases already used for Ragas and DeepEval. LangSmith is added around the application; the RAG is not rebuilt in LangChain.

Created by Toni Ramchandani

linkedin.com/in/toni-ramchandani

1. The entire idea in one page

A working RAG can answer a question, and Ragas or DeepEval can score selected qualities. The remaining operational problem is understanding, reproducing, comparing, and learning from complete application executions. LangSmith supplies that platform layer.



Question	What answers it here
What happened inside one request?	A trace containing root, retriever, and LLM runs.
Can the behavior be reproduced?	A governed dataset and an experiment over all eight examples.
Did a controlled change help?	Baseline and candidate experiments plus pairwise comparison.
Why did a score change?	Per-example outputs, evidence, evaluator comments, metadata, and trace details.
How do production failures improve testing?	Feedback and selected traces can feed new governed dataset examples.

Core principle
LangSmith does not make an evaluator correct. It executes, associates, visualizes, and preserves evaluation evidence. Metric definitions remain your responsibility.

2. What LangSmith is - and is not

LangSmith is an observability and evaluation platform for LLM applications. It records application execution as traces, organizes test examples as datasets, runs application versions as experiments, stores evaluator and human feedback, and supports analysis across versions.

It is framework-agnostic at the application boundary. This project uses the Python SDK's tracing functions around a plain Python RAG; it does not convert the RAG to LangChain.

LangSmith is	LangSmith is not
A system for tracing chains, retrieval, model calls, tools, and agents	A replacement for the application itself
A dataset and experiment management layer	A guarantee that a dataset is representative
A host for code, model-based, and human feedback	One universal quality metric
A comparison and investigation interface	Proof that a higher aggregate mean is safe
A bridge between offline evaluation and production observations	Automatic governance, privacy approval, or release policy

Why it exists
LLM failures are path-dependent. The final answer alone often cannot tell whether the cause was retrieval, prompt assembly, the model, latency, an evaluator, or bad reference data.

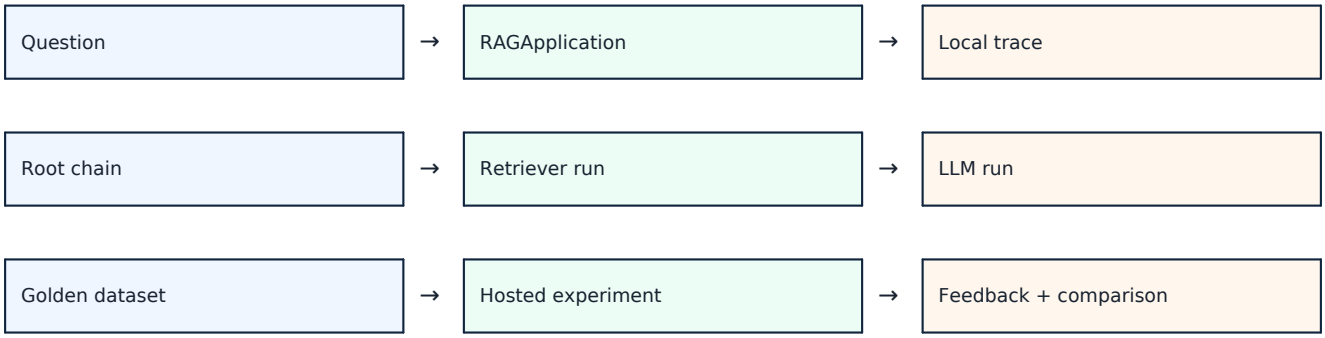
3. Where it fits after RAG, Ragas, and DeepEval

These layers solve related but different problems. The safest mental model is composition, not replacement.

Layer	Primary job in this project	Evidence produced
RAG application	Retrieve policy chunks and generate a grounded answer	Answer, retrieved contexts and IDs, model names, timings, local JSON trace
Ragas	Measure RAG-specific semantic relationships	Metric-specific scores produced by Ragas and its judge/embeddings
DeepEval	Represent cases as tests with thresholds, reasons, pass/fail, and G-Eval	DeepEval metric scores, reasons, errors, and threshold verdicts
LangSmith	Observe executions and manage datasets, experiments, feedback, comparison, and operational review	Traces, experiment rows, evaluator feedback, comparison records, and human labels

Do not merge unlike scores
A Ragas faithfulness score, a DeepEval faithfulness score, and a LangSmith feedback column are not interchangeable merely because all values fall between zero and one.

4. Architecture used by this project



Implementation boundary. LangSmithRAGApplication subclasses the existing RAGApplication. The base class still owns retrieval, prompt construction, generation, timings, and canonical local trace persistence. The subclass wraps the real ask(), _retrieve(), and _generate() boundaries with LangSmith tracing.

- No decorative post-hoc spans are fabricated after execution.
- Parent-child relationships are maintained through tracing context propagation.
- Provider errors continue to propagate while the active run records the failure.
- Hosted trace identity and local canonical trace identity remain distinct and linked.

5. The LangSmith object model

Object	Meaning	Project example
Project	Collection of operational traces	payroll-mfa-rag-observability
Run / span	One unit of work	payroll_mfa_retrieve
Trace	Root run plus descendants for one request	chain -> retriever + LLM
Dataset	Governed collection of examples	payroll-mfa-rag-golden-v1
Example	Inputs plus reference outputs and metadata	MFA-001
Experiment	One application configuration executed over a dataset	baseline-top3-*
Evaluator	Function that scores an output	required_context_recall
Feedback	Score, value, comment, or human review attached to a run	forbidden_claim_pass = 1
Comparative experiment	Pairwise preference over completed experiments	top3-vs-top5-*

Relationship
A dataset contains examples. An experiment runs one target configuration over those examples. Each experiment row has a trace and evaluator feedback. A comparative experiment compares aligned rows from completed experiments.

6. Code map: where each responsibility lives

Path	Responsibility
langsmith_app.py	CLI for preflight, tracing, dataset sync, experiments, comparison, re-evaluation, and feedback
langsmith_evaluation/settings.py	Endpoint, key, workspace, project, dataset, masking, and concurrency
langsmith_evaluation/tracing.py	Root chain, retriever, and LLM instrumentation around real RAG steps
langsmith_evaluation/dataset.py	Maps only the eight governed MFA cases into LangSmith examples
langsmith_evaluation/evaluators.py	Transparent retrieval, answer-policy, citation, exact-match, summary, and pairwise rules
langsmith_evaluation/runner.py	Dataset synchronization, live experiments, comparison, and cached re-evaluation
langsmith_evaluation/reporting.py	Local JSON and CSV evidence copy
evaluation/data/golden_cases.json	Single governed source for MFA-001 through MFA-008

Dependency boundary
The LangSmith implementation depends on the shared RAG and the governed MFA golden cases. Ragas and DeepEval remain separate evaluation implementations over the same application context.

7. Setup: account, Python, and model provider

Use Python 3.11 or 3.12 and the project's pinned dependencies. LangSmith authentication and model-provider authentication are separate.

```
# Windows PowerShell
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements-all-eval.txt
Copy-Item .env.example .env
```

Credential / service	Purpose	Needed when
LANGSMITH_API_KEY	Authorize hosted LangSmith operations	Tracing, dataset sync, hosted experiments, comparison, feedback, re-evaluation
Ollama server + models	Local embeddings and answer generation	--rag-provider ollama
OPENAI_API_KEY	OpenAI embeddings and answer generation	--rag-provider openai only

Secret handling
Keep keys only in the local .env or an approved secret manager. Never commit them, place them in screenshots, or paste them into shared material.

8. Configuration, explained

```
LANGSMITH_API_KEY=replace-locally
LANGSMITH_TRACING=true
LANGSMITH_ENDPOINT=https://api.smith.langchain.com
LANGSMITH_WORKSPACE_ID=
LANGSMITH_PROJECT=payroll-mfa-rag-observability
LANGSMITH_DATASET=payroll-mfa-rag-golden-v1
LANGSMITH_MAX_CONCURRENCY=2
LANGSMITH_CAPTURE_CONTENT=true
```

Variable	Meaning
LANGSMITH_ENDPOINT	API endpoint for the workspace region or deployment
LANGSMITH_WORKSPACE_ID	Explicit workspace routing; often blank for a PAT with one default workspace
LANGSMITH_PROJECT	Destination project for operational traces
LANGSMITH_DATASET	Hosted dataset selected by experiments
LANGSMITH_MAX_CONCURRENCY	Maximum parallel target/evaluator work
LANGSMITH_CAPTURE_CONTENT	Project-level switch controlling client masking of trace inputs and outputs

Synthetic workshop choice
Content capture can be enabled so the question, retrieved evidence, prompt, and answer are inspectable. Production data requires a separate classification, sanitization, access, retention, and regional-storage decision.

9. The workflow from zero to evidence

Step	Command	What changes
1	preflight --hosted	Read-only validation of configuration, eight cases, and API access
2	trace-one	Runs one real RAG request and writes one hosted trace
3	sync-dataset	Creates or updates eight stable hosted examples
4	run baseline	Executes one RAG configuration over all examples
5	run candidate	Executes one controlled variant over the same examples
6	compare	Creates pairwise evaluation over completed experiments; no target rerun
7	reevaluate	Applies changed evaluators to cached target runs
8	feedback	Attaches reviewed human or operational feedback to a run

Write boundary
Every hosted mutation requires --confirm-hosted. This makes uploads, model cost, endpoint, content capture, and retention an explicit operator decision.

10. Preflight: prove the control plane first

```
python langsmith_app.py preflight --hosted
```

Preflight validates the pinned SDK, all eight golden cases and chunk IDs, endpoint/workspace selection, hosted authentication through a read-only dataset listing, and the project/dataset/tracing configuration.

It validates	It does not do
Configuration parsing	Run Ollama or OpenAI
Golden-case integrity	Create or modify a dataset
Endpoint and workspace authorization	Write a trace
LangSmith API reachability	Call an evaluator or judge

Why this matters
A 401/403 at preflight is an authorization-routing problem. It should be fixed before mixing in model availability, retrieval, generation, or evaluation failures.

11. Trace one real RAG request

```
python langsmith_app.py trace-one `
  "I changed phones and cannot complete MFA. How do I regain payroll access?" `
  --rag-provider ollama `
  --top-k 3 `
  --confirm-hosted
```

The command executes the real RAG. It does not replay a frozen answer. The CLI flushes the SDK buffer before exit, resolves the hosted root run, and prints the local trace ID/path, hosted run ID/URL, project name, retrieved chunk IDs, and capture status.

What this creates

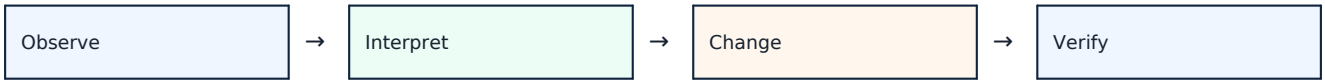
The trace turns one answer into inspectable evidence: the request, configuration, retrieved chunks, prompt, model output, latency, and any failure are tied to one execution. The value is not the tree itself; it is the ability to explain why the answer happened.

12. What we observe - and why it matters

Observation	What it tells us	Value created
Root status, answer, top_k and model metadata	Which configuration produced this outcome and whether the request completed	Reproduce an incident instead of arguing from screenshots
Retrieved chunk IDs, order, scores and text	Whether required policy evidence entered the prompt	Send the fix to retrieval, chunking, embeddings or corpus ownership
Constructed messages and answer	Whether the model received the evidence but omitted, distorted or contradicted it	Target prompt and generation changes without rebuilding retrieval
Retriever, LLM and total latency	Where time was spent	Optimize the bottleneck and evaluate the quality-latency trade-off
Errors and missing evaluator feedback	Whether failure came from the application, provider or evaluator	Keep infrastructure faults separate from legitimate quality failures
Tags and experiment metadata	Provider, model, top_k, dataset version and evaluator profile	Make results filterable, reproducible and auditable

The key distinction
A score says that something failed. The trace shows where the failure entered the execution. LangSmith is valuable because it keeps both pieces of evidence connected.

13. The observation-to-value chain



Observed in LangSmith	Interpretation	Action	Resulting value
MFA-001 trace lacks SEC-17::device-re-enrolment	The model never received all mandatory recovery evidence	Investigate ranking, chunking, query formulation, embeddings or top_k	Fix the evidence path; do not tune the answer prompt blindly
Both MFA-001 chunks are present, but successful test sign-in is omitted	Retrieval succeeded; generation did not use all required evidence	Change prompt/instructions or model, then rerun the same case	A smaller, attributable change with lower regression risk
MFA-002 forbidden_claim_pass = 0	The answer permits a prohibited bypass pattern	Treat as a blocking policy failure and inspect its LLM span	Prevent a severe failure from being hidden by a good average
MFA-006 is correct but LLM latency dominates	Quality is acceptable; generation is the performance bottleneck	Compare a faster model or generation configuration on the same dataset	Improve response time without confusing speed with correctness

These are diagnostic branches
The rows explain how to interpret observations if they appear in a live run. They are not claims that these outcomes occurred in a hosted experiment.

14. One bad answer, four different root causes

Suppose MFA-001 produces an incomplete recovery answer. The final text alone does not identify the correct fix. Open the trace and follow the evidence.

What the trace shows	Root-cause hypothesis	Correct next move
Required device re-enrolment chunk is absent	Retrieval, chunking, indexing, query or top_k problem	Inspect ranked results and retrieval scores; test one retrieval change
Required chunks are present, but the prompt omits one	Prompt assembly or context-window problem	Fix prompt construction or context selection
Prompt contains the policy, but the answer omits test sign-in	Generation instruction-following or model problem	Tighten instructions or compare a candidate model
Answer is correct, but evaluator reports missing concept	Evaluator regex may not recognize a valid paraphrase	Review the answer and evaluator comment; change the evaluator, then re-evaluate cached runs

Value
LangSmith reduces misdirected engineering work. It helps avoid changing embeddings when the model ignored good context, or changing the model when the evaluator itself produced the false alarm.

15. A practical investigation inside LangSmith

Order	Inspect	Decision question
1	Root run status, input, output and metadata	Which request and configuration failed?
2	Retriever child run and ranked documents	Were all mandatory chunks retrieved?
3	LLM child run and constructed messages	Did the required evidence reach the model?
4	Generated answer	Did the model use the evidence faithfully and completely?
5	Evaluator feedback and comment	Which explicit contract failed, and is the rule itself valid?
6	Sibling experiment row or comparison diff	Did the controlled candidate fix this case without breaking another?

For MFA-002, start with forbidden_claim_pass. If it is zero, inspect the matched pattern and answer, then verify whether the prohibited-shortcuts chunk entered the prompt. For MFA-008, inspect closure evidence and the successful-test-sign-in requirement before accepting a superficially helpful answer.

Stop condition
Do not move from observation to code change until the trace identifies the failing boundary or reveals that the evaluator/reference contract needs review.

16. The governed dataset

The hosted dataset is synchronized from evaluation/data/golden_cases.json. It is governed input to the experiment, not a dataset generated from traces at runtime.

Case	Question	Required chunks	Status
MFA-001	I changed phones and cannot complete MFA. How do I regain payroll acc...	2	approved
MFA-002	Can my manager approve a temporary MFA bypass because payroll closes ...	2	approved
MFA-003	I cannot complete MFA and payroll closes in three hours. How should t...	2	approved
MFA-004	What information must the Help Desk record in an MFA recovery ticket?	1	approved
MFA-005	My phone was stolen and I received an unexpected MFA prompt. What mus...	2	approved
MFA-006	Will payroll access definitely be restored within 30 minutes?	1	approved
MFA-007	What is the Help Desk phone number for payroll MFA recovery?	1	approved
MFA-008	An assisted payroll change is complete, but the new-device test sign-...	1	approved

Version rule
Changing source policy, chunk IDs, references, or expected behavior requires a dataset-version decision. Reusing the same version label after a semantic contract change weakens experiment comparability.

17. How one golden case becomes a LangSmith example

Example field	Stored content	Why it exists
inputs	case_id and question	The target receives the same governed request
outputs.answer	Approved reference answer	Reference-bounded comparison and inspection
outputs.required_context_ids	Mandatory evidence chunks	Deterministic retrieval recall
outputs.required_concepts	Alternative regex groups for required answer content	Transparent policy-coverage check
outputs.forbidden_claim_patterns	Disallowed policy claims	Blocking safety rule
outputs.expected_citation_ids	Expected inline chunk IDs	Citation recall
metadata	Case ID, tags, source, version	Filtering, provenance, and governed analysis

Example IDs are deterministic UUID5 values derived from the dataset version and MFA case ID. Repeated synchronization upserts stable examples instead of intentionally creating duplicates.

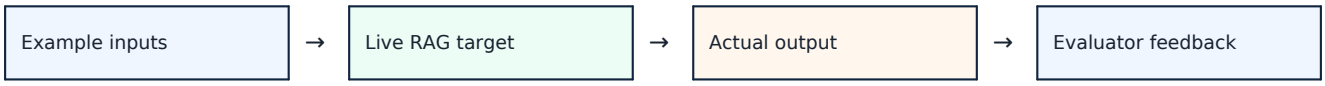
```
python langsmith_app.py sync-dataset --confirm-hosted
```

Privacy correction
Trace masking and dataset upload are separate. LANGSMITH_CAPTURE_CONTENT=false can hide trace inputs/outputs, but sync-dataset still uploads the dataset payload selected by the code.

18. What an experiment actually executes

```
python langsmith_app.py run `
  --rag-provider ollama `
  --top-k 3 `
  --metric-profile full `
  --hosted `
  --confirm-hosted `
  --experiment-prefix baseline-top3
```

For each hosted example, LangSmith passes example.inputs to the target. The target calls the real RAG and returns answer, retrieved contexts and IDs, models, latencies, and local trace identifiers. Evaluators receive that actual output plus example.outputs and attach feedback to the experiment run.



Experiment meaning
An experiment is not a metric. It is a complete application configuration executed over a dataset, with outputs, traces, metadata, and one or more evaluator results.

19. Evaluators in this implementation

Evaluator	Definition	Main limitation
required_context_recall	Mandatory chunk IDs retrieved / all mandatory chunk IDs	Does not judge ranking quality beyond presence
required_concept_coverage	Required answer-concept groups matched / all required groups	Regexes can miss valid paraphrases
forbidden_claim_pass	0 when a forbidden claim pattern matches; otherwise 1	A pass means no configured pattern matched, not universal safety
citation_validity	Cited chunk IDs that were retrieved / all cited chunk IDs	No citations needs explicit interpretation
citation_recall	Expected citation IDs found / all expected IDs	Requires an inline citation contract
exact_reference_match	Normalized actual answer equals normalized reference	Poor semantic metric for valid paraphrases

The core profile uses retrieval recall, concept coverage, and forbidden-claim pass. The full profile adds citation validity, citation recall, and exact reference match.

Evaluator ownership
These are deterministic project evaluators recorded through LangSmith. They are not Ragas metrics, DeepEval metrics, or LangSmith LLM-as-a-judge scores.

20. How to read an experiment correctly

Start here	Then inspect	Decision value
Run errors and missing feedback	Provider exceptions, evaluator errors, incomplete runs	Separate infrastructure failure from legitimate low quality
Blocking policy feedback	Every row where forbidden_claim_pass = 0	A severe case can override a passing average
Per-example score changes	Answer, retrieved chunks, reference contract, evaluator comment	Explain why a version changed
Trace timing	Retriever, LLM, and total latency	Find performance regressions and bottlenecks
Aggregate means	Distribution, repetitions, groups, and worst cases	Summarize only after row-level safety review

Use compact view for scanning, full view for outputs, and diff view when comparing generated text with references. Open a row to inspect the complete trace alongside feedback.

No invented numbers
This guide does not contain a fixed experiment score. Ollama output is live and can vary; the current run must be inspected and measured in LangSmith.

21. Baseline and candidate: make a fair change

A useful comparison changes one intended variable while keeping the dataset version, corpus, prompt, model, evaluator code, and repetition count fixed.

```
# Baseline
python langsmith_app.py run --rag-provider ollama --top-k 3 `
  --metric-profile full --hosted --confirm-hosted `
  --experiment-prefix baseline-top3

# Candidate: only top_k changes
python langsmith_app.py run --rag-provider ollama --top-k 5 `
  --metric-profile full --hosted --confirm-hosted `
  --experiment-prefix candidate-top5
```

Controlled	Confounded
top_k 3 vs top_k 5 with everything else fixed	top_k, model, prompt, and dataset all changed together
Same evaluator version on both experiments	New evaluator logic applied only to the candidate
Same repetition count and concurrency intent	One version measured once and another measured repeatedly

Non-determinism
Use --num-repetitions 3 when variance matters. Eight cases then create 24 target runs, increasing time and model cost but reducing dependence on one lucky sample.

22. Pairwise comparison

```
python langsmith_app.py compare `
  "EXACT_BASELINE_EXPERIMENT_NAME" `
  "EXACT_CANDIDATE_EXPERIMENT_NAME" `
  --experiment-prefix top3-vs-top5 `
  --confirm-hosted
```

The command compares two completed experiments over aligned examples. It does not rerun the RAG. Output order is randomized to reduce systematic first-position preference.

Decision order	Rule used here
1. Safety	Prefer the run that passes the forbidden-claim gate
2. Evidence quality	If safety is equal, compare mean mandatory-context recall, required-concept coverage, and citation recall
3. Tie	Record equal preference when the vectors are equal

Important limitation
Pairwise preference tells which of two supplied outputs is better under this rule. It does not prove either output meets an absolute release threshold.

23. Re-evaluation versus a new experiment

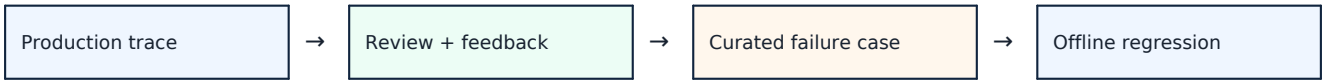
```
python langsmith_app.py reevaluate `
  "EXPERIMENT_NAME" `
  --metric-profile full `
  --confirm-hosted
```

Situation	Correct action	Why
Evaluator regex or scoring logic changed	Re-evaluate cached experiment	Target outputs are still valid; only interpretation changed
New retriever, prompt, model, corpus, or top_k	Run a new experiment	Application behavior must be executed again
Reference contract changed	Version dataset, then run a new experiment	Old and new ground truth should not be silently mixed

Evidence integrity
Re-evaluation is cheaper because it avoids target execution, but it cannot tell how a changed application would behave.

24. Human feedback and the production loop

```
python langsmith_app.py feedback RUN_ID `
  --key human_correctness `
  --score 1 `
  --comment "Approved after reviewing answer and evidence" `
  --confirm-hosted
```



Feedback becomes useful evidence only when its rubric and provenance are clear. In production, record reviewer identity, rubric version, review context, timestamp, and whether retention was intentionally extended.

- Do not treat a bare thumbs-up as equivalent to policy correctness.
- Sample common behavior, but deliberately capture rare high-risk incidents.
- Promote reviewed failures into versioned dataset examples before changing the system.
- Validate the fix offline, compare it with the baseline, then deploy and continue monitoring.

25. Where the practical value comes from

Capability	Decision it improves	Practical value
Nested traces	Which component should be changed?	Less blind tuning and faster root-cause isolation
Governed datasets	Can the behavior be reproduced across known obligations?	Repeatable regression evidence instead of ad hoc prompts
Per-example evaluator feedback	Which requirement failed and why?	Specific defects with comments, not one unexplained average
Baseline/candidate experiments	Did a controlled change improve the system?	Evidence-based model, prompt and retrieval decisions
Pairwise diffs and blocking rules	Did the candidate introduce a severe regression?	Safer release or rollback decisions
Human feedback linked to runs	Which production failures deserve investigation?	Real incidents become governed future tests
Latency, error and usage observations	Is the system operationally acceptable?	Reliability and performance are evaluated with answer quality

The dashboard alone creates no value. Value appears only when an observation changes a decision: assign the defect to the right layer, reject an unsafe release, choose a better configuration, or promote a reviewed incident into the regression dataset.

For this payroll-MFA RAG
The strongest value is traceable policy assurance: every important answer can be connected to the evidence retrieved, the model step that used it, the evaluator contract applied, and the experiment in which a proposed change was tested.

26. Hosted and local modes

Capability	Local dry run	Hosted LangSmith
LangSmith API key	Not required	Required
RAG provider	Still required	Still required
Target and evaluators execute	Yes	Yes
upload_results	False	True
Persistent dataset/experiment UI	No	Yes
Local JSON/CSV report	Yes	Yes
Best use	Code-path check and isolated case debugging	Teaching, collaboration, experiment history, and operational review

```
# One-case local SDK execution with no LangSmith upload
python langsmith_app.py run `
  --rag-provider ollama `
  --case-id MFA-001 `
  --top-k 3 `
  --metric-profile core
```

Meaning
A local dry run verifies the SDK evaluation path. It is not evidence that hosted authentication, tracing, dataset persistence, or collaboration features work.

27. Privacy, security, retention, and cost

Risk	Production control
Sensitive prompt, context, or answer content	Classify and sanitize data; mask trace content; restrict project access
Dataset upload contains governed examples	Review dataset payload separately from trace masking
Wrong regional endpoint or workspace	Verify deployment region, workspace ID, key scope, and role before upload
Credential exposure	Rotate immediately; store in secret manager; never log or commit
Long retention from feedback/evaluation	Set retention deliberately and avoid extending it by default
High experiment cost	Control cases, repetitions, concurrency, judge calls, and provider model
Rare safety failures missed by sampling	Keep blocking deterministic checks and targeted incident sampling

Project-specific limitation
The custom Ollama provider does not automatically expose complete token-usage or model-cost metadata. Measure those separately if cost accounting is required.

28. Failure diagnosis

Symptom	Likely cause	First check
401/403 during preflight	Invalid key, wrong endpoint, workspace routing, or role	Effective endpoint, key type/scope, and workspace ID
Tracing must be true	LANGSMITH_TRACING=false when process started	Enable it and start a fresh shell/process
Trace exists but content is hidden	Client masking active	LANGSMITH_CAPTURE_CONTENT and data approval
Dataset exists but run cannot find it	Different workspace/endpoint/name	Use the same effective .env for sync and run
No nested spans	Wrong code path or instrumentation disabled	Use trace-one and inspect exact run names
Target errors	Ollama/OpenAI, model, index, or timeout failure	Open failing run and local provider exception
Feedback blank	Evaluator exception or missing reference output	Open evaluator error; do not convert blank to zero
Comparison rows misalign	Experiments do not share governed examples	Same dataset and version

Debugging order
Control plane first, model provider second, target execution third, evaluator execution fourth, analysis last. Mixing these layers hides the actual failure.

29. What this workflow proves - and what it cannot

It can prove	It cannot prove by itself
Which inputs, evidence, model step, output, and timing belonged to a run	That the retrieved source was factually current or authorized
How one configuration performed on eight governed examples	Generalization to unrepresented production traffic
Whether deterministic project rules passed on each output	Complete semantic correctness or absence of all unsafe paraphrases
Which of two completed outputs a stated pairwise rule prefers	That the preferred output clears an absolute quality floor
Whether evaluator changes alter judgments on cached outputs	How a changed application would execute
Where reviewers attached feedback	That labels are reliable without rubric, calibration, and provenance

Release decision
Use blocking safety rules, per-case evidence, aggregate trends, model variance, human review, and production risk together. Do not release from one dashboard mean.

30. Complete live runbook

```
# 1. Validate hosted access
python langsmith_app.py preflight --hosted

# 2. Observe one live request
python langsmith_app.py trace-one "MFA recovery question" `
  --rag-provider ollama --top-k 3 --confirm-hosted

# 3. Upload the eight governed examples
python langsmith_app.py sync-dataset --confirm-hosted

# 4. Run baseline and candidate experiments
python langsmith_app.py run --rag-provider ollama --top-k 3 `
  --metric-profile full --hosted --confirm-hosted `
  --experiment-prefix baseline-top3
python langsmith_app.py run --rag-provider ollama --top-k 5 `
  --metric-profile full --hosted --confirm-hosted `
  --experiment-prefix candidate-top5

# 5. Compare exact returned experiment names
python langsmith_app.py compare "BASELINE_NAME" "CANDIDATE_NAME" `
  --experiment-prefix top3-vs-top5 --confirm-hosted
```

After each step, inspect the returned URL before continuing. Validate the trace tree, dataset example contract, experiment errors, feedback columns, changed rows, and worst-case policy behavior.

Completion condition
The workflow is complete when the baseline and candidate are traceable, reproducible against the same dataset version, explainable row by row, and compared under an explicit decision rule.

31. Practical next steps

Next improvement	Why it matters
Add dataset splits and metadata groups	Analyze recovery, urgency, compromise, and audit cases separately
Calibrate deterministic rules with human labels	Measure known misses and false alarms instead of assuming regex completeness
Add an LLM-as-a-judge only for suitable subjective dimensions	Complement deterministic policy gates without replacing them
Create online evaluators with filters and sampling	Monitor deployed traffic while controlling cost and retention
Promote reviewed failures into a new dataset version	Turn production learning into repeatable regression evidence
Define release gates outside the dashboard	Keep severe-case policies explicit, versioned, and auditable

The strongest production pattern is a closed loop: observe real behavior, review failures, govern examples, test candidate changes offline, compare evidence, release deliberately, and observe again.

Final mental model
RAG produces behavior. Ragas and DeepEval measure selected qualities. LangSmith preserves the execution context and organizes the evidence needed to improve the system safely.

32. Official references and verification boundary

Official LangSmith documentation used to verify the concepts and workflow:

- Observability concepts: runs, traces, projects, feedback, metadata, and retention
- Evaluation concepts: datasets, examples, experiments, evaluators, and offline/online evaluation
- Custom instrumentation for non-LangChain application code
- Run an application over a dataset
- Pairwise evaluation of completed experiments
- Apply evaluators to existing experiment runs
- Attach user or human feedback
- Analyze experiment rows, traces, diffs, repetitions, and evaluator runs
- Compare experiment results, regressions, improvements, and side-by-side diffs
- Monitor trace count, latency, errors, model calls, tokens, and cost

Implementation verification
The project pins langsmith==0.10.17 and includes tests for the eight-case dependency boundary, a real local SDK evaluation with upload_results=False, dataset synchronization contract, trace preservation, deterministic evaluators, and pairwise API usage. Hosted results in this guide are intentionally not fabricated.